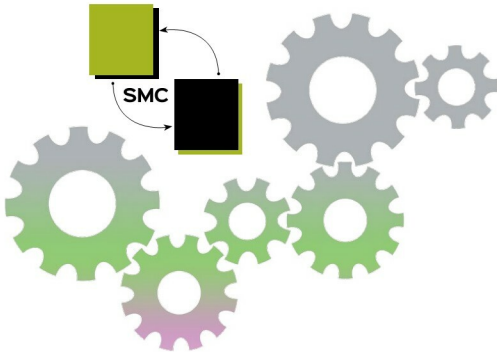


Using State Machines to Build Better Software

Often, an application runs in an environment of asynchronous, unordered and unpredictable events and the software needs to be robust enough to work seamlessly under these conditions. In this article, targeted at software designers and developers, we explore the benefits of using state machines to make robust software and the open source options available for this purpose.



Almost every software becomes complex as it evolves over a period of time, and can therefore suffer from frequent non-functional problems. These problems are linked to its maintainability, code manageability, extensibility, modifiability, determinism, etc. However, a good plan and design carved out at the initial stages could help developers enjoy the smoothness of the development process, as the software matures over a period of time.

The use of state machines is one of the oldest and best known techniques for modelling the behaviour of a system. And its adoption dates way back to the study of physical matter. As a reference, water (H₂O) can exist in three different states – solid, liquid and gaseous — and in each state it exhibits a specific behaviour; also, there are defined events and actions that lead to water, steam or ice transitioning through its different states. Software designers have been using this example to model and partition systems into well-defined states, with specific actions and events that define how these systems move through these states.

In this article, we look at the benefits of using state machines for modelling software along with some examples, and see what the open source world has to offer in this space.

Benefits of state machines

Some of the benefits of adopting state machines are listed below.

- A complex problem can be broken down into smaller and manageable pieces called states, with each state being delegated a narrowly defined task. Modelling the system with a state machine ensures a self-documenting and compact representation of the software that automatically satisfies non-functional demands like manageability, extensibility, modifiability and determinism. It might seem like a lot of work in the beginning but as the system grows, this technique helps to dramatically reduce the overall complexity. Enabling a state machine for code that is already developed and has evolved quite a bit can be painful. So, the one-time investment in state machines for any software module right at the beginning of the software